

第21章 USART—串口通讯

本章参考资料：《STM32F4xx 中文参考手册》USART 章节。

学习本章时，配合《STM32F4xx 中文参考手册》USART 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

特别说明，本书内容是以 STM32F4xxx 系列控制器资源讲解。

21.1 串口通讯协议简介

串口通讯(Serial Communication)是一种设备间非常常用的串行通讯方式，因为它简单便捷，大部分电子设备都支持该通讯方式，电子工程师在调试设备时也经常使用该通讯方式输出调试信息。

在计算机科学里，大部分复杂的问题都可以通过分层来简化。如芯片被分为内核层和片上外设；STM32 标准库则是在寄存器与用户代码之间的软件层。对于通讯协议，我们也以分层的方式来理解，最基本的是把它分为物理层和协议层。物理层规定通讯系统中具有机械、电子功能部分的特性，确保原始数据在物理媒体的传输。协议层主要规定通讯逻辑，统一收发双方的数据打包、解包标准。简单来说物理层规定我们用嘴巴还是用肢体来交流，协议层则规定我们用中文还是英文来交流。

下面我们分别对串口通讯协议的物理层及协议层进行讲解。

21.1.1 物理层

串口通讯的物理层有很多标准及变种，我们主要讲解 RS-232 标准，RS-232 标准主要规定了信号的用途、通讯接口以及信号的电平标准。

使用 RS-232 标准的串口设备间常见的通讯结构见图 21-1。

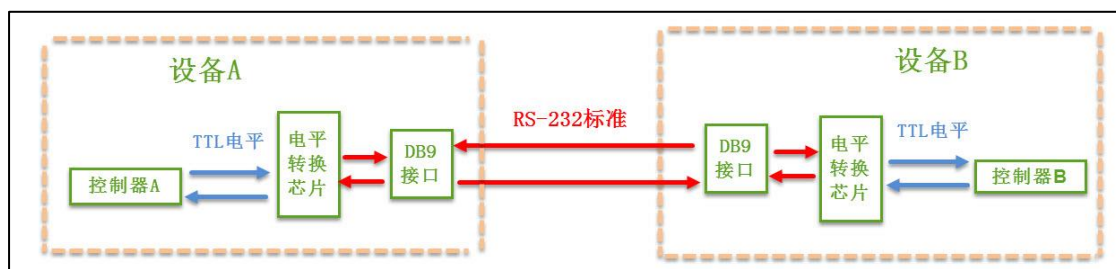


图 21-1 串口通讯结构图

在上面的通讯方式中，两个通讯设备的“DB9 接口”之间通过串口信号线建立起连接，串口信号线中使用“RS-232 标准”传输数据信号。由于 RS-232 电平标准的信号不能直接被控制器直接识别，所以这些信号会经过一个“电平转换芯片”转换成控制器能识别的“TTL 校准”的电平信号，才能实现通讯。

1. 电平标准

根据通讯使用的电平标准不同，串口通讯可分为 TTL 标准及 RS-232 标准，见表 21-1。

表 21-1 TTL 电平标准与 RS232 电平标准

通讯标准	电平标准(发送端)
5V TTL	逻辑 1: 2.4V-5V 逻辑 0: 0~0.5V
RS-232	逻辑 1: -15V~-3V 逻辑 0: +3V~+15V

我们知道常见的电子电路中常使用 TTL 的电平标准，理想状态下，使用 5V 表示二进制逻辑 1，使用 0V 表示逻辑 0；而为了增加串口通讯的远距离传输及抗干扰能力，它使用 -15V 表示逻辑 1，+15V 表示逻辑 0。使用 RS232 与 TTL 电平校准表示同一个信号时的对比见图 21-2。

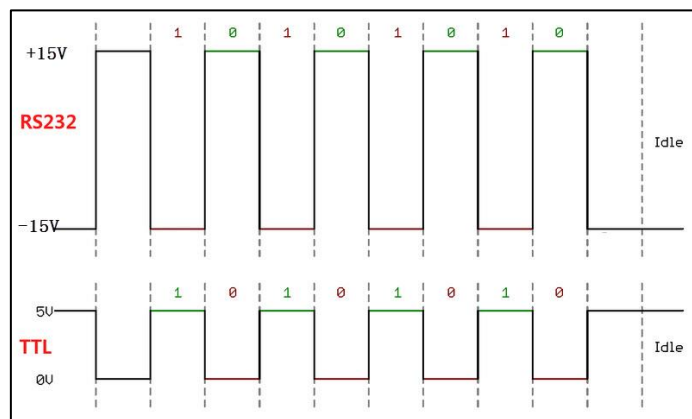


图 21-2 RS-232 与 TTL 电平标准下表示同一个信号

因为控制器一般使用 TTL 电平标准，所以常常会使用 MA3232 芯片对 TTL 及 RS-232 电平的信号进行互相转换。

2. RS-232 信号线

在最初的应用中，RS-232 串口标准常用于计算机、路由与调制解调器(MODEN，俗称“猫”)之间的通讯，在这种通讯系统中，设备被分为数据终端设备 DTE(计算机、路由)和数据通讯设备 DCE(调制解调器)。我们以这种通讯模型讲解它们的信号线连接方式及各个信号线的作用。

在旧式的台式计算机中一般会有 RS-232 标准的 COM 口(也称 DB9 接口)，见图 21-3。



图 21-3 电脑主板上的 COM 口及串口线

其中接线口以针式引出信号线的称为公头，以孔式引出信号线的称为母头。在计算机中一般引出公头接口，而在调制解调器设备中引出的一般为母头，使用上图中的串口线即可把它与计算机连接起来。通讯时，串口线中传输的信号就是使用前面讲解的 RS-232 标准调制的。

在这种应用场合下，DB9 接口中的公头及母头的各个引脚的标准信号线接法见图 21-4 及表 21-2。

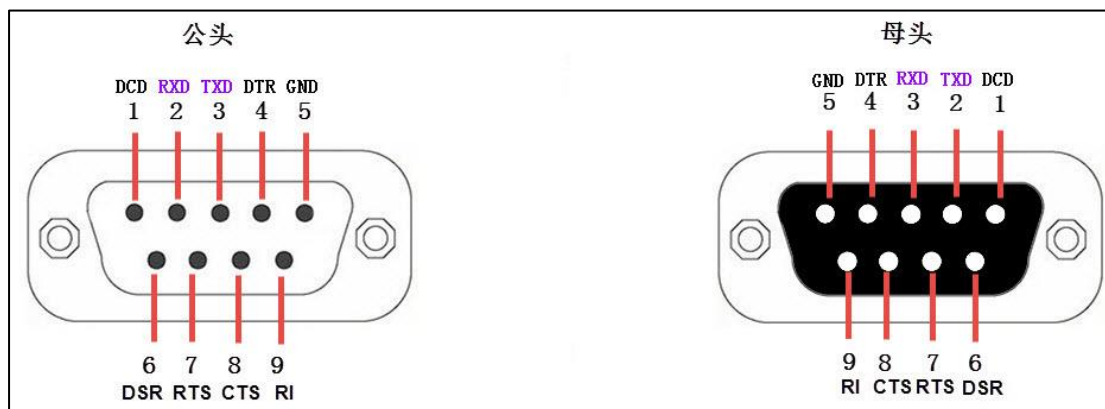


图 21-4 DB9 标准的公头及母头接法

表 21-2 DB9 信号线说明(公头，为方便理解，可把 DTE 理解为计算机，DCE 理解为调制调

解器)

序号	名称	符号	数据方向	说明
1	载波检测	DCD	DTE→DCE	Data Carrier Detect, 数据载波检测, 用于 DTE 告知对方, 本机是否收到对方的载波信号
2	接收数据	RXD	DTE←DCE	Receive Data, 数据接收信号, 即输入。
3	发送数据	TXD	DTE→DCE	Transmit Data, 数据发送信号, 即输出。两个设备之间的 TXD 与 RXD 应交叉相连
4	数据终端 (DTE) 就绪	DTR	DTE→DCE	Data Terminal Ready, 数据终端就绪, 用于 DTE 向对方告知本机是否已准备好
5	信号地	GND	-	地线, 两个通讯设备之间的地电位可能不一样, 这会影响收发双方的电平信号, 所以两个串口设备之间必须要使用地线连接, 即共地。
6	数据设备 (DCE) 就绪	DSR	DTE←DCE	Data Set Ready, 数据发送就绪, 用于 DCE 告知对方本机是否处于待命状态
7	请求发送	RTS	DTE→DCE	Request To Send, 请求发送, DTE 请求 DCE 本设备向 DCE 端发送数据
8	允许发送	CTS	DTE←DCE	Clear To Send, 允许发送, DCE 回应对方的 RTS 发送请求, 告知对方是否可以发送数据
9	响铃指示	RI	DTE←DCE	Ring Indicator, 响铃指示, 表示 DCE 端与线路已接通

上表中的是计算机端的 DB9 公头标准接法, 由于两个通讯设备之间的收发信号(RXD 与 TXD)应交叉相连, 所以调制解调器端的 DB9 母头的收发信号接法一般与公头的相反, 两个设备之间连接时, 只要使用“直通型”的串口线连接起来即可, 见图 21-5。

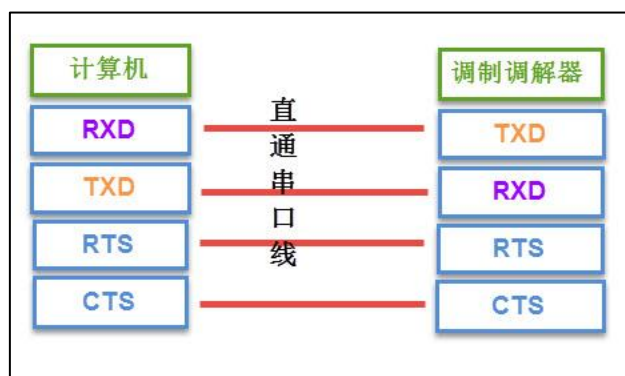


图 21-5 计算机与调制解调器的信号线连接

串口线中的 RTS、CTS、DSR、DTR 及 DCD 信号, 使用逻辑 1 表示信号有效, 逻辑 0 表示信号无效。例如, 当计算机端控制 DTR 信号线表示为逻辑 1 时, 它是为了告知远端的调制解调器, 本机已准备好接收数据, 0 则表示还没准备就绪。

在目前的其它工业控制使用的串口通讯中, 一般只使用 RXD、TXD 以及 GND 三条信号线, 直接传输数据信号。而 RTS、CTS、DSR、DTR 及 DCD 信号都被裁剪掉了, 如果您在前面被这些信号弄得晕头转向, 那就直接忽略它们吧。

21.1.2 协议层

串口通讯的数据包由发送设备通过自身的 TXD 接口传输到接收设备的 RXD 接口。在串口通讯的协议层中，规定了数据包的内容，它由起始位、主体数据、校验位以及停止位组成，通讯双方的数据包格式要约定一致才能正常收发数据，其组成见图 21-6。

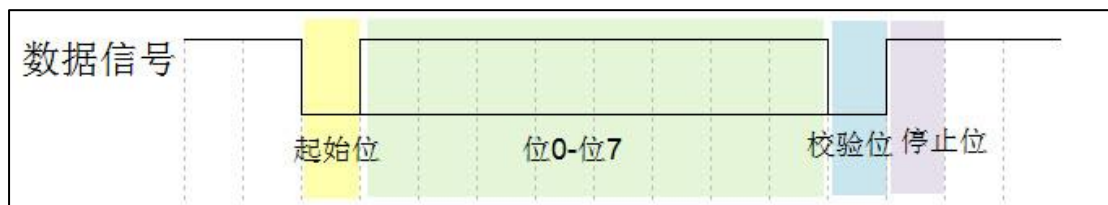


图 21-6 串口数据包的基本组成

1. 波特率

本章中主要讲解的是串口异步通讯，异步通讯中由于没有时钟信号(如前面讲解的 DB9 接口中是没有时钟信号的)，所以两个通讯设备之间需要约定好波特率，即每个码元的长度，以便对信号进行解码，图 21-6 中用虚线分开的每一格就是代表一个码元。常见的波特率为 4800、9600、115200 等。

2. 通讯的起始和停止信号

串口通讯的一个数据包从起始信号开始，直到停止信号结束。数据包的起始信号由一个逻辑 0 的数据位表示，而数据包的停止信号可由 0.5、1、1.5 或 2 个逻辑 1 的数据位表示，只要双方约定一致即可。

3. 有效数据

在数据包的起始位之后紧接着的就是要传输的主体数据内容，也称为有效数据，有效数据的长度常被约定为 5、6、7 或 8 位长。

4. 数据校验

在有效数据之后，有一个可选的数据校验位。由于数据通信相对更容易受到外部干扰导致传输数据出现偏差，可以在传输过程加上校验位来解决这个问题。校验方法有奇校验(odd)、偶校验(even)、0 校验(space)、1 校验(mark)以及无校验(noparity)，它们介绍如下：

- ❑ 奇校验要求有效数据和校验位中“1”的个数为奇数，比如一个 8 位长的有效数据为：01101001，此时总共有 4 个“1”，为达到奇校验效果，校验位为“1”，最后传输的数据将是 8 位的有效数据加上 1 位的校验位总共 9 位。

- 偶校验与奇校验要求刚好相反，要求帧数据和校验位中“1”的个数为偶数，比如数据帧：11001010，此时数据帧“1”的个数为4个，所以偶校验位为“0”。
- 0 校验是不管有效数据中的内容是什么，校验位总为“0”，1 校验是校验位总为“1”。
- 在无校验的情况下，数据包中不包含校验位。

21.2 STM32 的 USART 简介

STM32 芯片具有多个 USART 外设用于串口通讯，它是 Universal Synchronous Asynchronous Receiver and Transmitter 的缩写，即通用同步异步收发器可以灵活地与外部设备进行全双工数据交换。有别于 USART，它还有具有 UART 外设(Universal Asynchronous Receiver and Transmitter)，它是在 USART 基础上裁剪掉了同步通信功能，只有异步通信。简单区分同步和异步就是看通信时需不需要对外提供时钟输出，我们平时用的串口通信基本都是 UART。

USART 满足外部设备对工业标准 NRZ 异步串行数据格式的要求，并且使用了小数波特率发生器，可以提供多种波特率，使得它的应用更加广泛。USART 支持同步单向通信和半双工单线通信；还支持局域互连网络 LIN、智能卡(SmartCard)协议与 IrDA(红外线数据协会) SIR ENDEC 规范。

USART 支持使用 DMA，可实现高速数据通信，有关 DMA 具体应用将在 DMA 章节作具体讲解。

USART 在 STM32 应用最多莫过于“打印”程序信息，一般在硬件设计时都会预留一个 USART 通信接口连接电脑，用于在调试程序是可以把一些调试信息“打印”在电脑端的串口调试助手工具上，从而了解程序运行是否正确、指出运行出错位置等等。

STM32 的 USART 输出的是 TTL 电平信号，若需要 RS-232 标准的信号可使用 MAX3232 芯片进行转换。

21.3 USART 功能框图

STM32 的 USART 功能框图包含了 USART 最核心内容，掌握了功能框图，对 USART 就有一个整体的把握，在编程时就思路就非常清晰，见图 21-7。

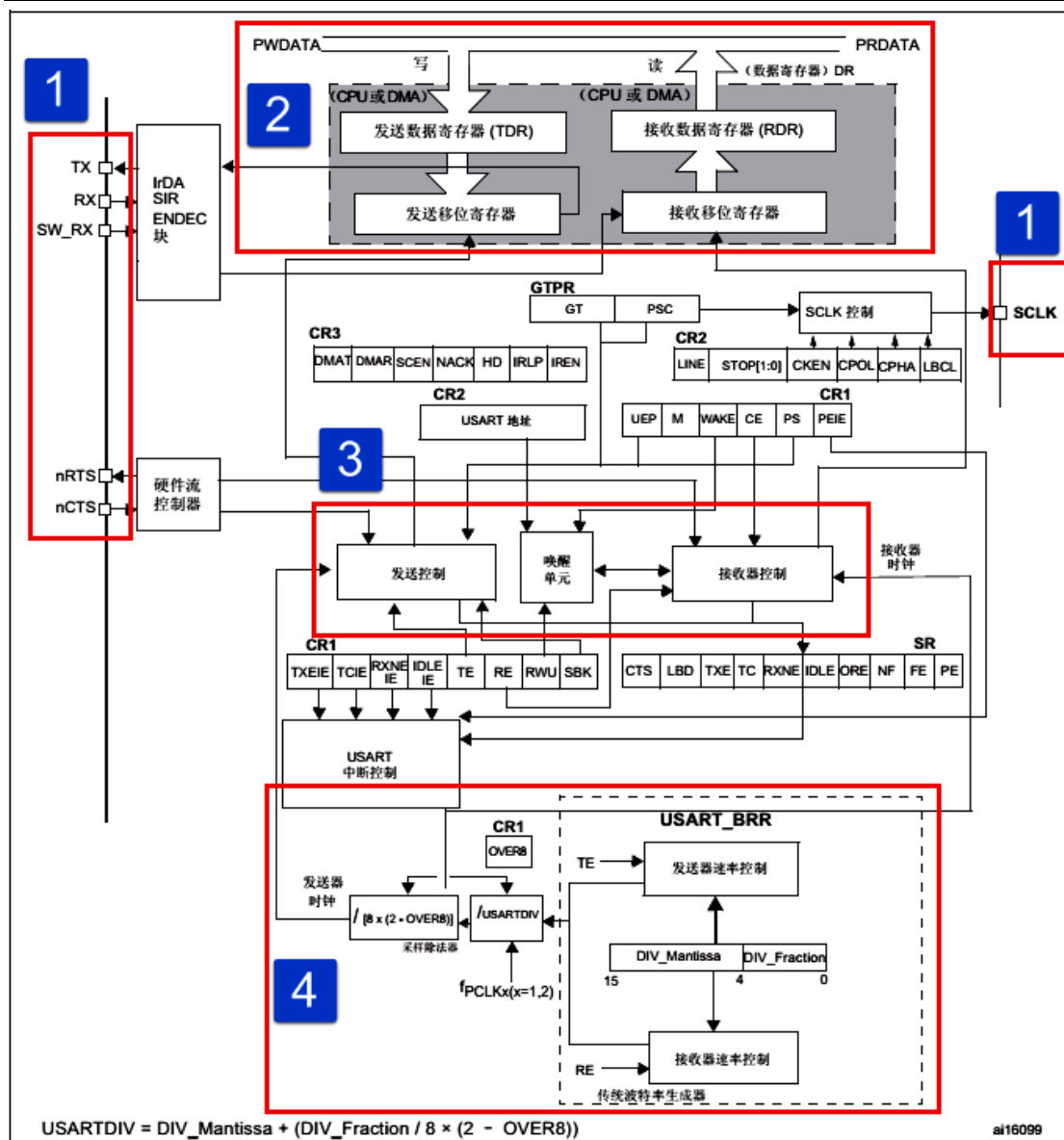


图 21-7 USART 功能框图

1. ①功能引脚

TX: 发送数据输出引脚。

RX: 接收数据输入引脚。

SW_RX: 数据接收引脚，只用于单线和智能卡模式，属于内部引脚，没有具体外部引脚。

nRTS: 请求以发送(Request To Send)，n 表示低电平有效。如果使能 RTS 流控制，当 USART 接收器准备好接收新数据时就会将 nRTS 变成低电平；当接收寄存器已满时，nRTS 将被设置为高电平。该引脚只适用于硬件流控制。

nCTS: 清除以发送(Clear To Send), n 表示低电平有效。如果使能 CTS 流控制, 发送器在发送下一帧数据之前会检测 nCTS 引脚, 如果为低电平, 表示可以发送数据, 如果为高电平则在发送完当前数据帧之后停止发送。该引脚只适用于硬件流控制。

SCLK: 发送器时钟输出引脚。这个引脚仅适用于同步模式。

USART 引脚在 STM32F407ZGT6 芯片具体发布见表 21-3。

表 21-3 STM32F407ZGT6 芯片的 USART 引脚

	APB2(最高 84MHz)		APB1(最高 42MHz)			
	USART1	USART6	USART2	USART3	UART4	UART5
TX	PA9/PB6	PC6/PG14	PA2/PD5	PB10/PD8 /PC10	PA0/PC10	PC12
RX	PA10/PB7	PC7/PG9	PA3/PD6	PB11/PD9 /PC11	PA1/PC11	PD2
SCLK	PA8	PG7/PC8	PA4/PD7	PB12/PD10 /PC12	-	-
nCTS	PA11	PG13/PG15	PA0/PD3	PB13/PD11	-	-
nRTS	PA12	PG8/PG12	PA1/PD4	PB14/PD12	-	-

STM32F407ZGT6 有四个 USART 和两个 UART, 其中 USART1 和 USART6 的时钟来源于 APB2 总线时钟, 其最大频率为 84MHz, 其他四个的时钟来源于 APB1 总线时钟, 其最大频率为 42MHz。

UART 只是异步传输功能, 所以没有 SCLK、nCTS 和 nRTS 功能引脚。

观察表 21-3 可发现很多 USART 的功能引脚有多个引脚可选, 这非常方便硬件设计, 只要在程序编程时软件绑定引脚即可。

2. ②数据寄存器

USART 数据寄存器(USART_DR)只有低 9 位有效, 并且第 9 位数据是否有效要取决于 USART 控制寄存器 1(USART_CR1)的 M 位设置, 当 M 位为 0 时表示 8 位数据字长, 当 M 位为 1 表示 9 位数据字长, 我们一般使用 8 位数据字长。

USART_DR 包含了已发送的数据或者接收到的数据。USART_DR 实际是包含了两个寄存器, 一个专门用于发送的可写 TDR, 一个专门用于接收的可读 RDR。当进行发送操作时, 往 USART_DR 写入数据会自动存储在 TDR 内; 当进行读取操作时, 向 USART_DR 读取数据会自动提取 RDR 数据。

TDR 和 RDR 都是介于系统总线和移位寄存器之间。串行通信是一个位一个位传输的, 发送时把 TDR 内容转移到发送移位寄存器, 然后把移位寄存器数据每一位发送出去, 接收时把接收到的每一位顺序保存在接收移位寄存器内然后才转移到 RDR。

USART 支持 DMA 传输, 可以实现高速数据传输, 具体 DMA 使用将在 DMA 章节讲解。

3. ③控制器

USART 有专门控制发送的发送器、控制接收的接收器，还有唤醒单元、中断控制等等。使用 USART 之前需要向 USART_CR1 寄存器的 UE 位置 1 使能 USART。发送或者接收数据字长可选 8 位或 9 位，由 USART_CR1 的 M 位控制。

发送器

当 USART_CR1 寄存器的发送使能位 TE 置 1 时，启动数据发送，发送移位寄存器的数据会在 TX 引脚输出，如果是同步模式 SCLK 也输出时钟信号。

一个字符帧发送需要三个部分：起始位+数据帧+停止位。起始位是一个位周期的低电平，位周期就是每一位占用的时间；数据帧就是我们要发送的 8 位或 9 位数据，数据是从最低位开始传输的；停止位是一定时间周期的高电平。

停止位时间长短是可以通过 USART 控制寄存器 2(USART_CR2)的 STOP[1:0]位控制，可选 0.5 个、1 个、1.5 个和 2 个停止位。默认使用 1 个停止位。2 个停止位适用于正常 USART 模式、单线模式和调制解调器模式。0.5 个和 1.5 个停止位用于智能卡模式。

当选择 8 位字长，使用 1 个停止位时，具体发送字符时序图见图 21-8。

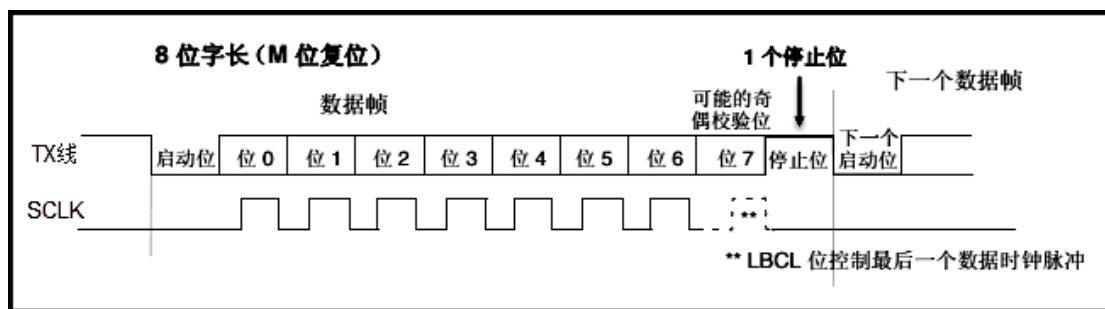


图 21-8 字符发送时序图

当发送使能位 TE 置 1 之后，发送器开始会先发送一个空闲帧(一个数据帧长度的高电平)，接下来就可以往 USART_DR 寄存器写入要发送的数据。在写入最后一个数据后，需要等待 USART 状态寄存器(USART_SR)的 TC 位为 1，表示数据传输完成，如果 USART_CR1 寄存器的 TCIE 位置 1，将产生中断。

在发送数据时，编程的时候有几个比较重要的标志位我们来总结下。

名称	描述
TE	发送使能
TXE	发送寄存器为空，发送单个字节的时候使用
TC	发送完成，发送多个字节数据的时候使用
TXIE	发送完成中断使能

接收器

如果将 USART_CR1 寄存器的 RE 位置 1，使能 USART 接收，使得接收器在 RX 线开始搜索起始位。在确定到起始位后就根据 RX 线电平状态把数据存放在接收移位寄存器内。

接收完成后就把接收移位寄存器数据移到 RDR 内，并把 USART_SR 寄存器的 RXNE 位置 1，同时如果 USART_CR2 寄存器的 RXNEIE 置 1 的话可以产生中断。

在接收数据时，编程的时候有几个比较重要的标志位我们来总结一下。

名称	描述
RE	接收使能
RXNE	读数据寄存器非空
RXNEIE	发送完成中断使能

为得到一个信号真实情况，需要用一个比这个信号频率高的采样信号去检测，称为过采样，这个采样信号的频率大小决定最后得到源信号准确度，一般频率越高得到的准确度越高，但为了得到越高频率采样信号越也困难，运算和功耗等等也会增加，所以一般选择合适就好。

接收器可配置为不同过采样技术，以实现从噪声中提取有效的数据。USART_CR1 寄存器的 OVER8 位用来选择不同的采样方法，如果 OVER8 位设置为 1 采用 8 倍过采样，即用 8 个采样信号采样一位数据；如果 OVER8 位设置为 0 采用 16 倍过采样，即用 16 个采样信号采样一位数据。

USART 的起始位检测需要用到特定序列。如果在 RX 线识别到该特定序列就认为是检测到了起始位。起始位检测对使用 16 倍或 8 倍过采样的序列都是一样的。该特定序列为：1110X0X0000，其中 X 表示电平任意，1 或 0 皆可。

8 倍过采样速度更快，最高速度可达 $f_{PCLK}/8$ ， f_{PCLK} 为 USART 时钟，采样过程见图 21-9。使用第 4、5、6 次脉冲的值决定该位的电平状态。

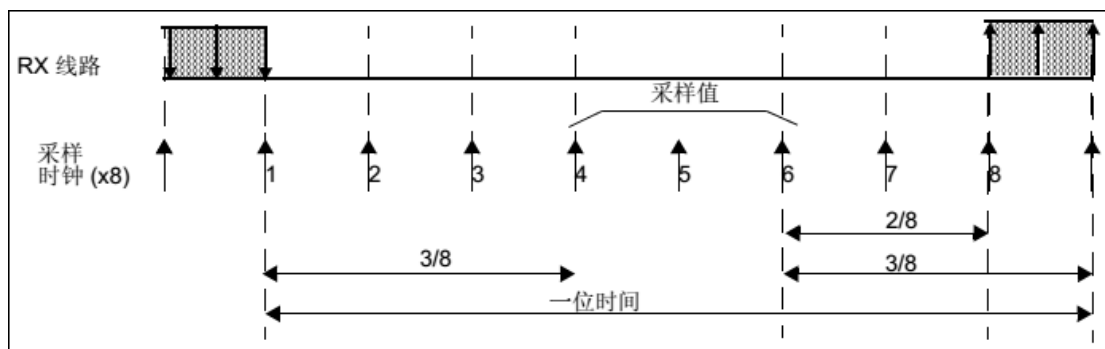


图 21-9 8 倍过采样过程

16 倍过采样速度虽然没有 8 倍过采样那么快，但得到的数据更加精准，其最大速度为 $f_{PCLK}/16$ ，采样过程见图 21-10。使用第 8、9、10 次脉冲的值决定该位的电平状态。

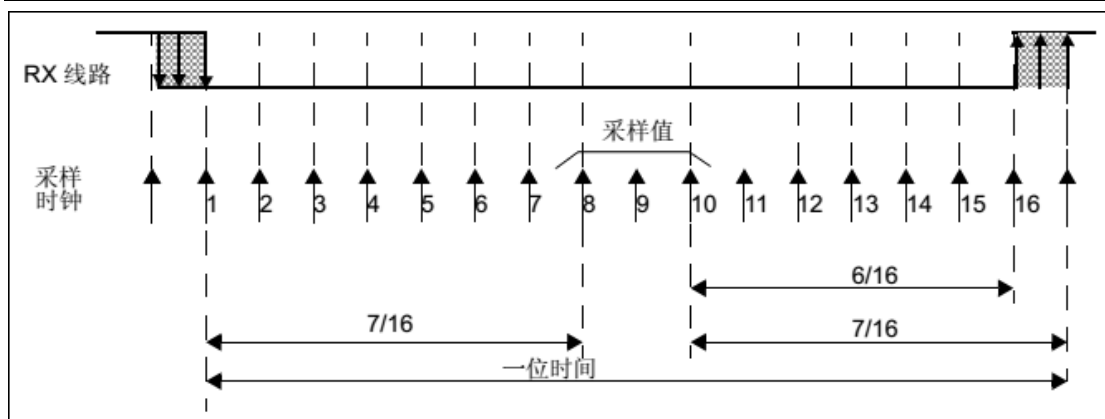


图 21-10 16 倍过采样过程

4. ④小数波特率生成

波特率指数据信号对载波的调制速率，它用单位时间内载波调制状态改变次数来表示，单位为波特。比特率指单位时间内传输的比特数，单位 bit/s(bps)。对于 USART 波特率与比特率相等，以后不区分这两个概念。波特率越大，传输速率越快。

USART 的发送器和接收器使用相同的波特率。计算公式如下：

$$\text{波特率} = \frac{f_{\text{PLCK}}}{8 \times (2 - \text{OVER8}) \times \text{USARTDIV}}$$

公式 21-1 波特率计算

其中， f_{PLCK} 为 USART 时钟，参考表 21-3；OVER8 为 USART_CR1 寄存器的 OVER8 位对应的值，USARTDIV 是一个存放在波特率寄存器(USART_BRR)的一个无符号定点数。其中 DIV_Mantissa[11:0]位定义 USARTDIV 的整数部分，DIV_Fraction[3:0]位定义 USARTDIV 的小数部分，DIV_Fraction[3]位只有在 OVER8 位为 0 时有效，否则必须清零。

例如，如果 OVER8=0，DIV_Mantissa=24 且 DIV_Fraction=10，此时 USART_BRR 值为 0x18A；那么 USARTDIV 的小数位 10/16=0.625；整数位 24，最终 USARTDIV 的值为 24.625。

如果 OVER8=0 并且知道 USARTDIV 值为 27.68，那么 DIV_Fraction=16*0.68=10.88，最接近的正整数为 11，所以 DIV_Fraction[3:0]为 0xB；DIV_Mantissa=整数(27.68)=27，即位 0x1B。

如果 OVER8=1 情况类似，只是把计算用到的权值由 16 改为 8。

波特率的常用值有 2400、9600、19200、115200。下面以实例讲解如何设定寄存器值得到波特率的值。

由表 21-3 可知 USART1 和 USART6 使用 APB2 总线时钟，最高可达 84MHz，其他 USART 的最高频率为 42MHz。我们选取 USART1 作为实例讲解，即 $f_{\text{PLCK}}=84\text{MHz}$ 。

当我们使用 16 倍过采样时即 OVER8=0，为得到 115200bps 的波特率，此时：

$$115200 = \frac{84000000}{8 * 2 * \text{USARTDIV}}$$

解得 USARTDIV=45.57，可算得 DIV_Fraction=0x9（ $0.57*2^4=9.12$ 取整，在 BRR 寄存器中，表示小数位的有 4 位），DIV_Mantissa=0x2D，即应该设置 USART_BRR 的值为 0x2D9。

在计算 DIV_Fraction 时经常出现小数情况，经过我们取舍得到整数，这样会导致最终输出的波特率较目标值略有偏差。下面我们从 USART_BRR 的值为 0x2D9 开始计算得出实际输出的波特率大小。

由 USART_BRR 的值为 0x2D9，可得 DIV_Fraction=45，DIV_Mantissa=9，所以 USARTDIV=45+9/16=45.5625，所以实际波特率为：115226；这个值跟我们的目标波特率相差很小，这么小的误差在正常通信的允许范围内。

8 倍过采样时计算情况原理是一样的。

5. 校验控制

STM32F4xx 系列控制器 USART 支持奇偶校验。当使用校验位时，串口传输的长度将是 8 位的数据帧加上 1 位的校验位总共 9 位，此时 USART_CR1 寄存器的 M 位需要设置为 1，即 9 数据位。将 USART_CR1 寄存器的 PCE 位置 1 就可以启动奇偶校验控制，奇偶校验由硬件自动完成。启动了奇偶校验控制之后，在发送数据帧时会自动添加校验位，接收数据时自动验证校验位。接收数据时如果出现奇偶校验位验证失败，会见 USART_SR 寄存器的 PE 位置 1，并可以产生奇偶校验中断。

使能了奇偶校验控制后，每个字符帧的格式将变成：起始位+数据帧+校验位+停止位。

6. 中断控制

USART 有多个中断请求事件，具体见表 21-4。

表 21-4 USART 中断请求

中断事件	事件标志	使能控制位
发送数据寄存器为空	TXE	TXEIE
CTS 标志	CTS	CTSIE
发送完成	TC	TCIE
准备好读取接收到的数据	RXNE	RXNEIE
检测到上溢错误	ORE	

检测到空闲线路	IDLE	IDLEIE
奇偶校验错误	PE	PEIE
断路标志	LBD	LBDIE
多缓冲通信中的噪声标志、 上溢错误和帧错误	NF/ORE/FE	EIE

21.4 USART 初始化结构体详解

标准库函数对每个外设都建立了一个初始化结构体，比如 USART_InitTypeDef，结构体成员用于设置外设工作参数，并由外设初始化配置函数，比如 USART_Init()调用，这些设定参数将会设置外设相应的寄存器，达到配置外设工作环境的目的。

初始化结构体和初始化库函数配合使用是标准库精髓所在，理解了初始化结构体每个成员意义基本上就可以对该外设运用自如了。初始化结构体定义在 stm32f4xx_usart.h 文件中，初始化库函数定义在 stm32f4xx_usart.c 文件中，编程时我们可以结合这两个文件内注释使用。

USART 初始化结构体

```
1 typedef struct {
2     uint32_t USART_BaudRate;           // 波特率
3     uint16_t USART_WordLength;         // 字长
4     uint16_t USART_StopBits;           // 停止位
5     uint16_t USART_Parity;             // 校验位
6     uint16_t USART_Mode;               // USART 模式
7     uint16_t USART_HardwareFlowControl; // 硬件流控制
8 } USART_InitTypeDef;
```

- 1) USART_BaudRate: 波特率设置。一般设置为 2400、9600、19200、115200。标准库函数会根据设定值计算得到 USARTDIV 值，见公式 21-1，并设置 USART_BRR 寄存器值。
- 2) USART_WordLength: 数据帧字长，可选 8 位或 9 位。它设定 USART_CR1 寄存器的 M 位的值。如果没有使能奇偶校验控制，一般使用 8 数据位；如果使能了奇偶校验则一般设置为 9 数据位。
- 3) USART_StopBits: 停止位设置，可选 0.5 个、1 个、1.5 个和 2 个停止位，它设定 USART_CR2 寄存器的 STOP[1:0]位的值，一般我们选择 1 个停止位。
- 4) USART_Parity: 奇偶校验控制选择，可选 USART_Parity_No(无校验)、USART_Parity_Even(偶校验)以及 USART_Parity_Odd(奇校验)，它设定 USART_CR1 寄存器的 PCE 位和 PS 位的值。
- 5) USART_Mode: USART 模式选择，有 USART_Mode_Rx 和 USART_Mode_Tx，允许使用逻辑或运算选择两个，它设定 USART_CR1 寄存器的 RE 位和 TE 位。

- 6) USART_HardwareFlowControl: 硬件流控制选择, 只有在硬件流控制模式才有效, 可选有(1)使能 RTS、(2)使能 CTS、(3)同时使能 RTS 和 CTS、(4)不使能硬件流。

当使用同步模式时需要配置 SCLK 引脚输出脉冲的属性, 标准库使用一个时钟初始化结构体 USART_ClockInitTypeDef 来设置, 因此该结构体内容也只有在同步模式才需要设置。

USART 时钟初始化结构体

```
1 typedef struct {  
2     uint16_t USART_Clock;      // 时钟使能控制  
3     uint16_t USART_CPOL;      // 时钟极性  
4     uint16_t USART_CPHA;      // 时钟相位  
5     uint16_t USART_LastBit;   // 最尾位时钟脉冲  
6 } USART_ClockInitTypeDef;
```

- 1) USART_Clock: 同步模式下 SCLK 引脚上时钟输出使能控制, 可选禁止时钟输出 (USART_Clock_Disable)或开启时钟输出(USART_Clock_Enable); 如果使用同步模式发送, 一般都需要开启时钟。它设定 USART_CR2 寄存器的 CLKEN 位的值。
- 2) USART_CPOL: 同步模式下 SCLK 引脚上输出时钟极性设置, 可设置在空闲时 SCLK 引脚为低电平(USART_CPOL_Low)或高电平(USART_CPOL_High)。它设定 USART_CR2 寄存器的 CPOL 位的值。
- 3) USART_CPHA: 同步模式下 SCLK 引脚上输出时钟相位设置, 可设置在时钟第一个变化沿捕获数据(USART_CPHA_1Edge)或在时钟第二个变化沿捕获数据。它设定 USART_CR2 寄存器的 CPHA 位的值。USART_CPHA 与 USART_CPOL 配合使用可以获得多种模式时钟关系。
- 4) USART_LastBit: 选择在发送最后一个数据位的时候时钟脉冲是否在 SCLK 引脚输出, 可以是不输出脉冲 (USART_LastBit_Disable)、输出脉冲 (USART_LastBit_Enable)。它设定 USART_CR2 寄存器的 LBCL 位的值。

21.5 USART1 接发通信实验

USART 只需两根信号线即可完成双向通信, 对硬件要求低, 使得很多模块都预留 USART 接口来实现与其他模块或者控制器进行数据传输, 比如 GSM 模块, WIFI 模块、蓝牙模块等等。在硬件设计时, 注意还需要一根“共地线”。

我们经常使用 USART 来实现控制器与电脑之间的数据传输。这使得我们调试程序非常方便, 比如我们可以把一些变量的值、函数的返回值、寄存器标志位等等通过 USART 发送到串口调试助手, 这样我们可以非常清楚程序的运行状态, 当我们正式发布程序时再把这些调试信息去除即可。

我们不仅仅可以将数据发送到串口调试助手, 我们还可以在串口调试助手发送数据给控制器, 控制器程序根据接收到的数据进行下一步工作。

首先，我们来编写一个程序实现开发板与电脑通信，在开发板上电时通过 USART 发送一串字符串给电脑，然后开发板进入中断接收等待状态，如果电脑有发送数据过来，开发板就会产生中断，我们在中断服务函数接收数据，并马上把数据返回发送给电脑。

21.5.1 硬件设计

为利用 USART 实现开发板与电脑通信，需要用到一个 USB 转 USART 的 IC，我们选择 CH340G 芯片来实现这个功能，CH340G 是一个 USB 总线的转接芯片，实现 USB 转 USART、USB 转 IrDA 红外或者 USB 转打印机接口，我们使用其 USB 转 USART 功能。具体电路设计见图 21-11。

我们将 CH340G 的 TXD 引脚通过跳帽 J11 与 USART1 的 RX 引脚连接，CH340G 的 RXD 引脚通过跳帽 J12 与 USART1 的 TX 引脚连接。CH340G 芯片集成在开发板上，其地线(GND)已与控制器的 GND 连通。如果我们想使用串口 2，那么就可以把开发板中的默认连接的跳帽拔掉，然后用杜邦线连接 USART2 与 CH340G 的通信接口就可实现通信，这个就是用了跳帽的好处，不会把 CH340G 的通信引脚固定死。

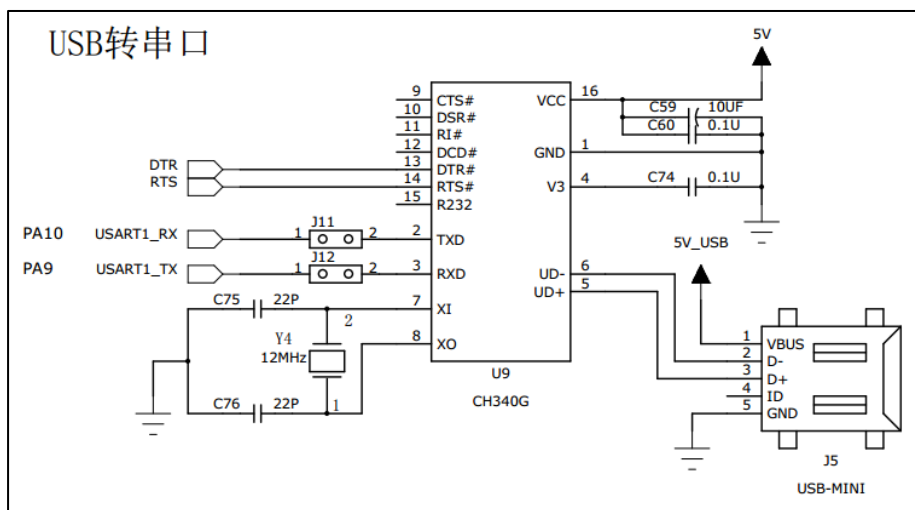


图 21-11 USB 转串口硬件设计

21.5.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：bsp_debug_usart.c 和 bsp_debug_usart.h 文件用来存放 USART 驱动程序及相关宏定义。

1. 编程要点

- 1) 使能 RX 和 TX 引脚 GPIO 时钟和 USART 时钟；
- 2) 初始化 GPIO，并将 GPIO 复用到 USART 上；

- 3) 配置 USART 参数;
- 4) 配置中断控制器并使能 USART 接收中断;
- 5) 使能 USART;
- 6) 在 USART 接收中断服务函数实现数据接收和发送。

2. 代码分析

GPIO 和 USART 宏定义

代码清单 21-1 GPIO 和 USART 宏定义

```
1 //引脚定义
2 /*****
3 #define DEBUG_USART                                USART1
4
5 /* 不同的串口挂载的总线不一样, 时钟使能函数也不一样, 移植时要注意
6 * 串口 1 和 6 是      RCC_APB2PeriphClockCmd
7 * 串口 2/3/4/5 是    RCC_APB1PeriphClockCmd
8 */
9
10
11 #define DEBUG_USART_CLK                            RCC_APB2Periph_USART1
12 #define DEBUG_USART_BAUDRATE                      115200 //串口波特率
13
14 #define DEBUG_USART_RX_GPIO_PORT                  GPIOA
15 #define DEBUG_USART_RX_GPIO_CLK                   RCC_AHB1Periph_GPIOA
16 #define DEBUG_USART_RX_PIN                        GPIO_Pin_10
17 #define DEBUG_USART_RX_AF                         GPIO_AF_USART1
18 #define DEBUG_USART_RX_SOURCE                     GPIO_PinSource10
19
20 #define DEBUG_USART_TX_GPIO_PORT                  GPIOA
21 #define DEBUG_USART_TX_GPIO_CLK                   RCC_AHB1Periph_GPIOA
22 #define DEBUG_USART_TX_PIN                        GPIO_Pin_9
23 #define DEBUG_USART_TX_AF                         GPIO_AF_USART1
24 #define DEBUG_USART_TX_SOURCE                     GPIO_PinSource9
25
26 #define DEBUG_USART_IRQHandler                    USART1_IRQHandler
27 #define DEBUG_USART_IRQ                           USART1_IRQn
28 /*****
```

使用宏定义方便程序移植和升级, 根据图 21-11 电路, 我们选择使用 USART1, 设定波特率为 115200, 一般我们会默认使用“8-N-1”参数, 即 8 个数据位、不用校验、一位停止位。查阅表 21-3 可知 USART1 的 TX 线可对于 PA9 和 PB6 引脚, RX 线可对于 PA10 和 PB7 引脚, 这里我们选择 PA9 以及 PA10 引脚。最后定义中断相关参数。

嵌套向量中断控制器 NVIC 配置

代码清单 21-2 中断控制器 NVIC 配置

```
1 static void NVIC_Configuration(void)
2 {
3     NVIC_InitTypeDef NVIC_InitStructure;
4
5     /* 嵌套向量中断控制器组选择 */
```

```
6     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);
7
8     /* 配置 USART 为中断源 */
9     NVIC_InitStructure.NVIC_IRQChannel = DEBUG_USART_IRQ;
10    /* 抢断优先级为 1 */
11    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
12    /* 子优先级为 1 */
13    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
14    /* 使能中断 */
15    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
16    /* 初始化配置 NVIC */
17    NVIC_Init(&NVIC_InitStructure);
18 }
19
```

在中断章节已对嵌套向量中断控制器的工作机制做了详细的讲解，这里我们就直接使用它，配置 USART 作为中断源，因为本实验没有使用其他中断，对优先级什么具体要求。

USART 初始化配置

代码清单 21-3 USART 初始化配置

```
1 void Debug_USART_Config(void)
2 {
3     GPIO_InitTypeDef GPIO_InitStructure;
4     USART_InitTypeDef USART_InitStructure;
5     /* 使能 USART GPIO 时钟 */
6     RCC_AHB1PeriphClockCmd(DEBUG_USART_RX_GPIO_CLK |
7                             DEBUG_USART_TX_GPIO_CLK,
8                             ENABLE);
9
10    /* 使能 USART 时钟 */
11    RCC_APB2PeriphClockCmd(DEBUG_USART_CLK, ENABLE);
12
13    /* 连接 Pxx 到 USARTx_Tx */
14    GPIO_PinAFConfig(DEBUG_USART_RX_GPIO_PORT,
15                    DEBUG_USART_RX_SOURCE,
16                    DEBUG_USART_RX_AF);
17
18    /* 连接 Pxx 到 USARTx_Rx */
19    GPIO_PinAFConfig(DEBUG_USART_TX_GPIO_PORT,
20                    DEBUG_USART_TX_SOURCE,
21                    DEBUG_USART_TX_AF);
22
23    /* GPIO 初始化 */
24    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
25    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
26    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
27
28    /* 配置 Tx 引脚为复用功能 */
29    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
30    GPIO_InitStructure.GPIO_Pin = DEBUG_USART_TX_PIN ;
31    GPIO_Init(DEBUG_USART_TX_GPIO_PORT, &GPIO_InitStructure);
32
33    /* 配置 Rx 引脚为复用功能 */
34    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
35    GPIO_InitStructure.GPIO_Pin = DEBUG_USART_RX_PIN;
36    GPIO_Init(DEBUG_USART_RX_GPIO_PORT, &GPIO_InitStructure);
37
38    /* 配置串 DEBUG_USART 模式 */
39    /* 波特率设置: DEBUG_USART_BAUDRATE */
40    USART_InitStructure.USART_BaudRate = DEBUG_USART_BAUDRATE;
41    /* 字长(数据位+校验位): 8 */
42    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
```

```
43      /* 停止位：1 个停止位 */
44      USART_InitStructure.USART_StopBits = USART_StopBits_1;
45      /* 校验位选择：不使用校验 */
46      USART_InitStructure.USART_Parity = USART_Parity_No;
47      /* 硬件流控制：不使用硬件流 */
48      USART_InitStructure.USART_HardwareFlowControl =
49          USART_HardwareFlowControl_None;
50      /* USART 模式控制：同时使能接收和发送 */
51      USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
52      /* 完成 USART 初始化配置 */
53      USART_Init(DEBUG_USART, &USART_InitStructure);
54
55      /* 嵌套向量中断控制器 NVIC 配置 */
56      NVIC_Configuration();
57
58      /* 使能串口接收中断 */
59      USART_ITConfig(DEBUG_USART, USART_IT_RXNE, ENABLE);
60
61      /* 使能串口 */
62      USART_Cmd(DEBUG_USART, ENABLE);
63 }
64
65
```

使用 GPIO_InitTypeDef 和 USART_InitTypeDef 结构体定义一个 GPIO 初始化变量以及一个 USART 初始化变量，这两个结构体内容我们之前已经有详细讲解。

调用 RCC_AHB1PeriphClockCmd 函数开启 GPIO 端口时钟，使用 GPIO 之前必须开启对应端口的时钟。使用 RCC_APB2PeriphClockCmd 函数开启 USART 时钟。

使用 GPIO 之前都需要初始化配置它，并且还要添加特殊设置，因为我们使用它作为外设的引脚，一般都有特殊功能。我们在初始化时需要把它的模式设置为复用功能。

每个 GPIO 都可以作为多个外设的特殊功能引脚，比如 PA10 这个引脚不仅仅可以作为普通的输入/输出引脚，还可以作为 USART1 的 RX 线引脚(USART1_RX)、定时器 1 通道 3 引脚(TIM1_CH3)、全速 OTG 的 ID 引脚(OTG_FS_ID)以及 DCMI 的数据 1 引脚(DCMI_D1)这四个外设的功能引脚，我们只能从中选择一个使用，这时就通过 GPIO 引脚复用功能配置(GPIO_PinAFConfig)函数实现复用功能引脚的连接。

这时我们可能会想如果程序把 PA10 用于 TIM1_CH3，此时 USART1_RX 就没办法使用了，那岂不是不能使用 USART1 了，实际上情况没有这么糟糕的，查阅表 21-3 我们可以看到 USART1_RX 不仅仅只有 PA10，还可以是 PB7。所以此时我们可以 PB7 这个引脚来实现 USART1 通信。那要是 PB7 也是被其他外设占用了呢？那就没办法了，只能使用其他 USART。

GPIO_PinAFConfig 函数接收三个参数，第一个参数为 GPIO 端口，比如 GPIOA；第二个参数是指定要复用的引脚号，比如 GPIO_PinSource10；第三个参数是选择复用外设，比如 GPIO_AF_USART1。该函数最终操作的是 GPIO 复用功能寄存器 GPIO_AFRH 和 GPIO_AFRL，分高低两个。

接下来，我们配置 USART1 通信参数并调用 USART 初始化函数完成配置。

程序用到 USART 接收中断，需要配置 NVIC，这里调用 NVIC_Configuration 函数完成配置。配置 NVIC 就可以调用 USART_ITConfig 函数使能 USART 接收中断。

最后调用 USART_Cmd 函数使能 USART。

字符发送

代码清单 21-4 字符发送函数

```
1  /***** 发送一个字符 *****/
2  void Usart_SendByte( USART_TypeDef * pUSARTx, uint8_t ch)
3  {
4      /* 发送一个字节数据到 USART */
5      USART_SendData(pUSARTx, ch);
6
7      /* 等待发送数据寄存器为空 */
8      while (USART_GetFlagStatus(pUSARTx, USART_FLAG_TXE) == RESET);
9  }
10
11 /***** 发送字符串 *****/
12 void Usart_SendString( USART_TypeDef * pUSARTx, char *str)
13 {
14     unsigned int k=0;
15     do {
16         Usart_SendByte( pUSARTx, *(str + k) );
17         k++;
18     } while (*(str + k) != '\0');
19
20     /* 等待发送完成 */
21     while (USART_GetFlagStatus(pUSARTx, USART_FLAG_TC) == RESET) {
22     }
23 }
```

Usart_SendByte 函数用来在指定 USART 发送一个 ASCII 码值字符，它有两个形参，第一个为 USART，第二个为待发送的字符。它是通过调用库函数 USART_SendData 来实现的，并且增加了等待发送完成功能。通过使用 USART_GetFlagStatus 函数来获取 USART 事件标志来实现发送完功能等待，它接收两个参数，一个是 USART，一个是事件标志。这里我们循环检测发送数据寄存器为空这个标志，当跳出 while 循环时说明发送数据寄存器为空这个事实。

Usart_SendString 函数用来发送一个字符串，它实际是调用 Usart_SendByte 函数发送每个字符，直到遇到空字符才停止发送。最后使用循环检测发送完成的事件标志来实现保证数据发送完成后才退出函数。

USART 中断服务函数

代码清单 21-5 USART 中断服务函数

```
1 void DEBUG_USART_IRQHandler(void)
2 {
3     uint8_t ucTemp;
4     if (USART_GetITStatus(DEBUG_USART, USART_IT_RXNE) != RESET) {
5         ucTemp = USART_ReceiveData( DEBUG_USART );
6         USART_SendData(DEBUG_USART, ucTemp);
7     }
8
9 }
```

这段代码是存放在 stm32f4xx_it.c 文件中的，该文件用来集中存放外设中断服务函数。当我们使能了中断并且中断发生时就会执行中断服务函数。

我们在代码清单 21-3 使能了 USART 接收中断，当 USART 有接收到数据就会执行 DEBUG_USART_IRQHandler 函数。USART_GetITStatus 函数与 USART_GetFlagStatus 函

数类似用来获取标志位状态，但 USART_GetITStatus 函数是专门用来获取中断事件标志的，并返回该标志位状态。使用 if 语句来判断是否是真的产生 USART 数据接收这个中断事件，如果是真的就使用 USART 数据读取函数 USART_ReceiveData 读取数据到指定存储区。然后再调用 USART 数据发送函数 USART_SendData 把数据又发送给源设备。

主函数

代码清单 21-6 主函数

```
1 #include "stm32f4xx.h"
2 #include "../usart/bsp_debug_usart.h"
3
4 /**
5  * @brief 主函数
6  * @param 无
7  * @retval 无
8  */
9 int main(void)
10 {
11     /*初始化 USART 配置模式为 115200 8-N-1, 中断接收*/
12     Debug_USART_Config();
13
14     /* 发送一个字符串 */
15     Usart_SendString( DEBUG_USART, "这是一个串口中断接收回显实验\n");
16
17     while (1) {
18     }
19 }
20 }
```

首先我们需要调用 Debug_USART_Config 函数完成 USART 初始化配置，包括 GPIO 配置，USART 配置，接收中断使用等等信息。

接下来就可以调用字符发送函数把数据发送给串口调试助手了。

最后主函数什么都不做，只是静静地等待 USART 接收中断的产生，并在中断服务函数把数据回传。

21.5.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板，此时串口调试助手即可收到开发板发过来的数据。我们在串口调试助手发送区域输入任意字符，点击发送按钮，马上在串口调试助手接收区即可看到相同的字符。



图 21-12 USART1 接发通信实验现象

21.6 USART1 指令控制 RGB 彩灯实验

在学习 C 语言时我们经常使用 C 语言标准函数库输入输出函数，比如 `printf`、`scanf`、`getchar` 等等。为让开发板也支持这些函数需要把 USART 发送和接收函数添加到这些函数的内部函数内。

正如之前所讲，可以在串口调试助手输入指令，让开发板根据这些指令执行一些任务，现在我们编写让程序接收 USART 数据，根据数据内容控制 RGB 彩灯的颜色。

21.6.1 硬件设计

硬件设计同第一个实验。

21.6.2 软件设计

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。我们创建了两个文件：`bsp_usart.c` 和 `bsp_usart.h` 文件用来存放 USART 驱动程序及相关宏定义。

1. 编程要点

- 1) 初始化配置 RGB 彩色灯 GPIO；
- 2) 使能 RX 和 TX 引脚 GPIO 时钟和 USART 时钟；
- 3) 初始化 GPIO，并将 GPIO 复用到 USART 上；

- 4) 配置 USART 参数;
- 5) 使能 USART;
- 6) 获取指令输入, 根据指令控制 RGB 彩色灯。

2. 代码分析

GPIO 和 USART 宏定义

代码清单 21-7 GPIO 和 USART 宏定义

```
1 //引脚定义
2 /*****
3 #define USARTx                                USART1
4
5 /* 不同的串口挂载的总线不一样, 时钟使能函数也不一样, 移植时要注意
6 * 串口 1 和 6 是      RCC_APB2PeriphClockCmd
7 * 串口 2/3/4/5 是    RCC_APB1PeriphClockCmd
8 */
9 #define USARTx_CLK                                RCC_APB2Periph_USART1
10 #define USARTx_CLOCKCMD                          RCC_APB2PeriphClockCmd
11 #define USARTx_BAUDRATE                          115200 //串口波特率
12
13 #define USARTx_RX_GPIO_PORT                      GPIOA
14 #define USARTx_RX_GPIO_CLK                      RCC_AHB1Periph_GPIOA
15 #define USARTx_RX_PIN                            GPIO_Pin_10
16 #define USARTx_RX_AF                            GPIO_AF_USART1
17 #define USARTx_RX_SOURCE                        GPIO_PinSource10
18
19 #define USARTx_TX_GPIO_PORT                      GPIOA
20 #define USARTx_TX_GPIO_CLK                      RCC_AHB1Periph_GPIOA
21 #define USARTx_TX_PIN                            GPIO_Pin_9
22 #define USARTx_TX_AF                            GPIO_AF_USART1
23 #define USARTx_TX_SOURCE                        GPIO_PinSource9
24
25 /*****/
```

使用宏定义方便程序移植和升级, 这里我们使用 USART1, 设定波特率为 115200。

USART 初始化配置

代码清单 21-8 USART 初始化配置

```
1 /**
2 * @brief USART GPIO 配置, 工作模式配置。115200 8-N-1
3 * @param 无
4 * @retval 无
5 */
6 void USARTx_Config(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9     USART_InitTypeDef USART_InitStructure;
10
11     RCC_AHB1PeriphClockCmd(USARTx_RX_GPIO_CLK |
12                             USARTx_TX_GPIO_CLK,
13                             ENABLE);
14
15     /* 使能 USART 时钟 */
16     USARTx_CLOCKCMD(USARTx_CLK, ENABLE);
```

```
17
18  /* 连接 Pxx 到 USARTx_Tx*/
19  GPIO_PinAFConfig(USARTx_RX_GPIO_PORT,
20                  USARTx_RX_SOURCE,
21                  USARTx_RX_AF);
22
23  /* 连接 Pxx 到 USARTx_Rx*/
24  GPIO_PinAFConfig(USARTx_TX_GPIO_PORT,
25                  USARTx_TX_SOURCE,
26                  USARTx_TX_AF);
27
28  /* GPIO 初始化 */
29  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
30  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
31  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
32
33  /* 配置 Tx 引脚为复用功能 */
34  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
35  GPIO_InitStructure.GPIO_Pin = USARTx_TX_PIN ;
36  GPIO_Init(USARTx_TX_GPIO_PORT, &GPIO_InitStructure);
37
38  /* 配置 Rx 引脚为复用功能 */
39  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
40  GPIO_InitStructure.GPIO_Pin = USARTx_RX_PIN;
41  GPIO_Init(USARTx_RX_GPIO_PORT, &GPIO_InitStructure);
42
43  /* 配置串 DEBUG_USART 模式 */
44  /* 波特率设置: DEBUG_USART_BAUDRATE */
45  USART_InitStructure.USART_BaudRate = USARTx_BAUDRATE;
46  /* 字长(数据位+校验位): 8 */
47  USART_InitStructure.USART_WordLength = USART_WordLength_8b;
48  /* 停止位: 1 个停止位 */
49  USART_InitStructure.USART_StopBits = USART_StopBits_1;
50  /* 校验位选择: 偶校验 */
51  USART_InitStructure.USART_Parity = USART_Parity_No;
52  /* 硬件流控制: 不使用硬件流 */
53  USART_InitStructure.USART_HardwareFlowControl =
54      USART_HardwareFlowControl_None;
55  /* USART 模式控制: 同时使能接收和发送 */
56  USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
57  /* 完成 USART 初始化配置 */
58  USART_Init(USARTx, &USART_InitStructure);
59
60  /* 使能串口 */
61  USART_Cmd(USARTx, ENABLE);
62 }
63
```

使用 GPIO_InitTypeDef 和 USART_InitTypeDef 结构体定义一个 GPIO 初始化变量以及一个 USART 初始化变量, 这两个结构体内容我们之前已经有详细讲解。

调用 RCC_AHB1PeriphClockCmd 函数开启 GPIO 端口时钟, 使用 GPIO 之前必须开启对应端口的时钟。

初始化配置 RX 线和 TX 线引脚为复用功能, 并将指定的 GPIO 连接至 USART1, 然后配置串口的工作参数为 115200-8-N-1。最后调用 USART_Cmd 函数使能 USART。

重定向 printf 和 scanf 函数

代码清单 21-9 重定向输入输出函数

```
1 ///重定向 c 库函数 printf 到串口, 重定向后可使用 printf 函数
2 int fputc(int ch, FILE *f)
```

```
3 {
4     /* 发送一个字节数据到串口 */
5     USART_SendData(USARTx, (uint8_t) ch);
6
7     /* 等待发送完毕 */
8     while (USART_GetFlagStatus(USARTx, USART_FLAG_TXE) == RESET);
9
10    return (ch);
11 }
12
13 ///重定向 c 库函数 scanf 到串口, 重写向后可使用 scanf、getchar 等函数
14 int fgetc(FILE *f)
15 {
16     /* 等待串口输入数据 */
17     while (USART_GetFlagStatus(USARTx, USART_FLAG_RXNE) == RESET);
18
19     return (int)USART_ReceiveData(USARTx);
20 }
```

在 C 语言标准库中, fputc 函数是 printf 函数内部的一个函数, 功能是将字符 ch 写入到文件指针 f 所指向文件的当前写指针位置, 简单理解就是把字符写入到特定文件中。我们使用 USART 函数重新修改 fputc 函数内容, 达到类似“写入”的功能。

fgetc 函数与 fputc 函数非常相似, 实现字符读取功能。在使用 scanf 函数时需要注意字符输入格式。

还有一点需要注意的, 使用 fputc 和 fgetc 函数达到重定向 C 语言标准库输入输出函数必须在 MDK 的工程选项把“Use MicroLIB”勾选上, MicroLIB 是缺省 C 库的备选库, 它对标准 C 库进行了高度优化使代码更少, 占用更少资源。

为使用 printf、scanf 函数需要在文件中包含 stdio.h 头文件。

输出提示信息

代码清单 21-10 输出提示信息

```
1 static void Show_Message(void)
2 {
3     printf("\r\n  这是一个通过串口通信指令控制 RGB 彩灯实验 \n");
4     printf("使用  USART1  参数为: %d 8-N-1 \n", USARTx_BAUDRATE);
5     printf("开发板接到指令后控制 RGB 彩灯颜色, 指令对应如下: \n");
6     printf("  指令  -----  彩灯颜色 \n");
7     printf("    1  -----  红 \n");
8     printf("    2  -----  绿 \n");
9     printf("    3  -----  蓝 \n");
10    printf("    4  -----  黄 \n");
11    printf("    5  -----  紫 \n");
12    printf("    6  -----  青 \n");
13    printf("    7  -----  白 \n");
14    printf("    8  -----  灭 \n");
15 }
```

Show_Message 函数全部是调用 printf 函数, “打印”实验操作信息到串口调试助手。

主函数

代码清单 21-11 主函数

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
```

```
3 #include "./usart/bsp_usart.h"
4
5 static void Show_Message(void);
6
7 /**
8  * @brief 主函数
9  * @param 无
10 * @retval 无
11 */
12 int main(void)
13 {
14     char ch;
15
16     /* 初始化 RGB 彩灯 */
17     LED_GPIO_Config();
18
19     /* 初始化 USART 配置模式为 115200 8-N-1 */
20     USARTx_Config();
21
22     /* 打印指令输入提示信息 */
23     Show_Message();
24     while (1) {
25         /* 获取字符指令 */
26         ch=getchar();
27         printf("接收到字符: %c\n",ch);
28
29         /* 根据字符指令控制 RGB 彩灯颜色 */
30         switch (ch) {
31             case '1':
32                 LED_RED;
33                 break;
34             case '2':
35                 LED_GREEN;
36                 break;
37             case '3':
38                 LED_BLUE;
39                 break;
40             case '4':
41                 LED_YELLOW;
42                 break;
43             case '5':
44                 LED_PURPLE;
45                 break;
46             case '6':
47                 LED_CYAN;
48                 break;
49             case '7':
50                 LED_WHITE;
51                 break;
52             case '8':
53                 LEDRGBOFF;
54                 break;
55             default:
56                 /* 如果不是指定指令字符, 打印提示信息 */
57                 Show_Message();
58                 break;
59         }
60     }
61 }
```

首先我们定义一个字符变量来存放接收到的字符。

接下来调用 LED_GPIO_Config 函数完成 RGB 彩色 GPIO 初始化配置, 该函数定义在 bsp_led.c 文件内。

调用 USARTx_Config 函数完成 USART 初始化配置。

Show_Message 函数使用 printf 函数打印实验指令说明信息。

getchar 函数用于等待获取一个字符，并返回字符。我们使用 ch 变量保持返回的字符，接下来判断 ch 内容执行对应的程序了。

我们使用 switch 语句判断 ch 变量内容，并执行对应的功能程序。

21.6.3 下载验证

保证开发板相关硬件连接正确，用 USB 线连接开发板“USB 转串口”接口跟电脑，在电脑端打开串口调试助手，把编译好的程序下载到开发板，此时串口调试助手即可收到开发板发过来的数据。我们在串口调试助手发送区域输入一个特定字符，点击发送按钮，RGB 彩色灯状态随之改变。